05 — Conclusão

> Quais diferenças o grupo percebeu entre desenvolver orientado por testes, especificar o comportamento e modelar o domínio?

As três técnicas atuaram em **níveis diferentes** do mesmo problema e se complementaram.

TDD mudou principalmente *a forma de escrever o código*. Escrever o teste antes obrigou a pensar em uma regra de cada vez e a implementar só o necessário. Na transferência, o teste de saldo insuficiente só falhou porque a implementação anterior era propositalmente mínima, e o teste de valor zero só falhou porque a correção do valor negativo tinha usado `< 0`. O ciclo também revelou dois defeitos reais que os testes manuais não pegavam: `setIdade()` ignorava a regra de idade e a transferência aceitava valores negativos. Outra percepção foi que, quando a regra já existe, o teste passa de primeira. Nesse caso ele funciona como teste de regressão, e não como TDD, e isso precisa ser dito com honestidade.

BDD mudou *a forma de descrever o que se espera*. Os cenários `Dado / Quando / Então` em português podem ser lidos por quem não programa e deixam claros casos que no código ficam implícitos, como "ativar um cliente que já está ativo" ou "consultar um cliente inexistente". Com o Cucumber, essa especificação também virou teste automatizado, ligando diretamente requisito, cenário e código.

DDD mudou *onde cada regra mora e como ela se chama*. Trocar o `boolean ativo` por `SituacaoCliente`/`SituacaoConta` e `setAtivo(true)` por `ativar()` aproximou o código da linguagem do negócio. Levar débito e crédito para dentro de `ContaCorrente` deixou o repositório responsável apenas por localizar as contas. DDD não substitui testes: foram os testes que garantiram que essas mudanças não quebraram nada.

Em resumo: **BDD** define *o que* o sistema deve fazer, **DDD** organiza *como o domínio é representado*, e **TDD** conduz *a implementação e a validação* passo a passo. Usadas juntas, como no requisito 12, cada técnica cobre uma lacuna das outras.